

Serviços e Interfaces para a Cloud - SIC_TSIW

UNIDADE CURRICULAR

Running microservices with orchestration tools

FICHA

Group work: This exercise should be completed in groups

Deadline: Submit on Moodle until 23h:59 of 26/11/2025

In this exercise, you'll deploy your microservices application using **Docker Swarm**, an orchestration tool that allows you to manage containers across multiple hosts as a single cluster. Docker Swarm simplifies **scaling, service discovery, load balancing**, and **fault tolerance**, making it a powerful solution for running distributed applications in production-like environments. Through this exercise, you'll gain hands-on experience with service orchestration and maintaining high availability within a microservices architecture.

1. Create a docker-compose-swarm.yml file to define your services.
2. Scale all services so there are no SPOFs in your application.
3. Run an API Gateway that makes use of Swarm's service discovery

You should submit your code and a screenshot of the `docker stack services` command output after you have everything running and working.

Using Docker swarm

First, you should create a `docker-compose-swarm.yml` file at the root of the directory where all your services are. Make sure you have all the necessary docker images built in your computer (you did this in the previous class). In this file you should define all the services that compose your application:

```
version: "3.8"

services:
  mongodb:
    image: mongo:8
    ports:
      - "27017:27017"
    environment:
      MONGO_INITDB_ROOT_USERNAME: root
      MONGO_INITDB_ROOT_PASSWORD: pass123
    networks:
      - micro-net
    deploy:
      replicas: 1

  movies-service:
    image: movies-service:1.0
    ports:
      - "3001:3001"
    networks:
      - micro-net
    deploy:
      replicas: 1

... #TODO: the rest of your services here

networks:
  micro-net:
    driver: overlay
    attachable: true
```

After you created the `docker-compose-swarm.yml` file you can run all your services using:

```
docker stack deploy -c docker-compose-swarm.yml movieflix-stack
```

Next, check if the services are running, with:

```
docker stack services movieflix-stack
```

In your browser check if you have access to all services.

Scaling services with Docker Swarm

In the `docker-compose-swarm.yml` file there is a “replicas” field in every service, which defines the default number of replicas for each service. If your stack is already running, you can increase or decrease the number of replicas using:

```
docker service scale <service_name>=<number_of_replicas>
```

Make sure all the services have at least 2 running containers, so it ensures high availability of the system.

Note: don't scale the database, keep it with only one replica because databases are usually not run in a container like this, this is just for testing purposes.

Now, check if you have multiple replicas of your services running with:

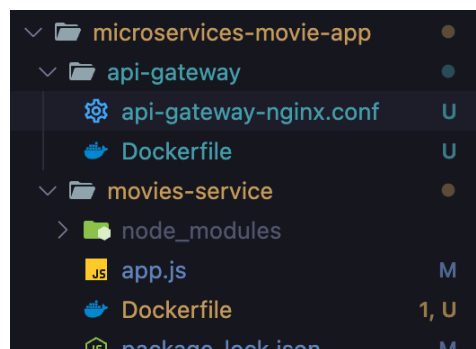
```
docker stack services movieflix-stack
```

Check in your browser that you have access to all services, **now docker swarm automatically load balances your requests across the multiple replicas**, with no need of further configurations.

Using an API Gateway as a single-entry point to the whole application

In one of the first classes you used an API Gateway to load balance requests across replicas of services and as a way of having a single-entry point/URL to access all services. Since our orchestration tool, Docker Swarm, deals with the load balancing across replicas for us, we only need to use an API Gateway to have a single-entry point to our microservices application.

To have an API Gateway to be part of our application, we need to build a custom NGINX docker image with our configuration file. For that, we need to create a **Dockerfile** for it:



```

# Use the official Nginx image
FROM nginx:alpine

# Remove the default configuration
RUN rm /etc/nginx/conf.d/default.conf

# Copy your custom Nginx configuration
COPY api-gateway-nginx.conf /etc/nginx/conf.d/api-gateway.conf

# Expose port 80 (Swarm will handle published ports)
EXPOSE 80

```

Since the API Gateway will now be running in a container and will be part of the docker network we defined, in the **api-gateway-nginx.conf** file we should update the URLs of the services like this:

```

server {
    listen 80;

    # Movies
    location /api/movies {
        proxy_pass http://movies-service:3001/movies;
    }

    # Reviews
    location /api/reviews {
        proxy_pass http://reviews-service:3002/reviews;
    }

    # Users
    location /api/users {
        proxy_pass http://users-service:3003/users;
    }
}

```

After we define the Dockerfile and the api-gatway-nginx.conf, we can build the docker image for the API Gateway:

```
docker build -t api-gateway:1.0 .
```

Don't forget to stop your stack before we add the new service for the API Gateway:

```
docker stack rm movieflix-stack
```

Now we have to add the API Gateway to our `docker-compose-swarm.yml` file, so it runs along with our services:

```
api-gateway:
  image: api-gateway:1.0
  ports:
    - "80:80"
  networks:
    - micro-net
  deploy:
    replicas: 1
```

After all the changes, you can run the swarm stack again:

```
docker stack deploy -c docker-compose-swarm.yml movieflix-stack
```

Check if the API Gateway is running and try to access all the services from the API Gateway:

- localhost:80/api/movies
- localhost:80/api/reviews
- localhost:80/api/user